

MARC

Mascareignes Assistant and Robot Compagnon

Guide de reprise du projet

Laboratoire de robotique — Université des Mascareignes

Document destiné à la personne qui reprend le projet.

Sommaire

Sommaire.....	2
1. Le projet.....	4
Ce qui fonctionne	4
Code présent mais abandonné.....	4
Ce qui reste à faire.....	4
2. Les 6 pièges à connaître.....	5
3. Architecture	6
Les 8 nœuds ROS 2	6
Règle à respecter	6
Diagnostic	6
4. Communication Raspberry Pi ↔ Arduino	7
Pi vers Arduino.....	7
Arduino vers Pi.....	7
Tester le lien sans ROS.....	7
5. Orientation au démarrage	8
Le repère de la salle.....	8
Pourquoi l'orientation compte	8
Procédure correcte	8
Re-tarer le cap	8
Recalage par vision	8
6. Installation	10
Raspberry Pi.....	10
Arduino	10
Chemins codés en dur	10
Calibration caméra	10
7. Démarrage et utilisation	12
Checklist.....	12
Lancement	12
Interface web.....	12
Le micro physique.....	12
Actions disponibles	13
8. Modifier la carte	14
Ajouter ou déplacer une station.....	14
Ajouter un point de passage.....	14
Ajouter un marqueur ArUco.....	14

9. Modifier les yeux	15
Ajouter une expression.....	15
Ajouter un style complet	15
Réglages du panneau.....	15
10. Réglages de navigation	16
11. Dépannage.....	17
12. Aide-mémoire	18
Fichiers les plus souvent modifiés	18

1. Le projet

MARC est un robot guide autonome du laboratoire de robotique. Il se déplace seul entre les stations, dialogue en langage naturel grâce à un modèle de langage (LLM), et exprime des émotions sur une matrice LED. Le tout est supervisé depuis une interface web temps réel.

Login Raspberry pi : ssh pi@11.255.255.119 avec le mot de passe : 123456.
raspi119

Mécanique entièrement modélisée sur Onshape et imprimée en 3D au laboratoire :

Modèle 3D complet : [ouvrir sur Onshape](#)

Les fichiers STL et STEP sont également dans le dépôt, sous assets/3d-models/.

Ce qui fonctionne

- Navigation autonome vers une station (vision ArUco + odométrie)
- Planification de trajet contournant les obstacles fixes (Dijkstra)
- Commandes en langage naturel interprétées par LLM
- Interface web temps réel : carte, pose, caméra, Push-To-Talk, journal
- Matrice LED 64×32 avec expressions animées, synthèse vocale française

Code présent mais abandonné

Ne perdez pas de temps dessus :

Élément	Pourquoi
Suivi de ligne (capteurs IR)	Remplacé par la navigation par vision. Le code Arduino répond encore à M:1.
Mode auto-équilibrant (PID)	Conflit avec la navigation : deux contrôleurs se disputent les mêmes moteurs.
web/server.py	Ancien serveur monolithique, remplacé par webbridge_node.py.
vision/*.py	Prototypes hors ROS, portés en nœuds. Utiles pour tester une brique isolée.

Ce qui reste à faire

- Mesurer et renseigner les positions des stations pepper et imp3d
- Passer le STT en local (Whisper.cpp ou Vosk) pour supprimer la dépendance à Google
- Remplacer le micro embarqué, puis réactiver voice_node
- Ajouter un watchdog série côté Arduino (sécurité)

2. Les 6 pièges à connaître

Ces points ne se devinent pas en lisant le code. Ils vous feront perdre du temps.

#	Piège	Détail
1	Orientation de départ imposée	Le cap est mis à zéro 5 s après le démarrage. Mal orienté = navigation faussée, sans message d'erreur. Voir §4.
2	Micro physique désactivé	Sa qualité est trop mauvaise. On pilote par le Push-To-Talk de l'interface web. Voir §7.
3	Chemins codés en dur	Quatre nœuds pointent vers /home/pi/MARC. Cloner ailleurs casse tout. Voir §5.
4	Deux serveurs HTTPS	Ports 5000 (interface) et 5001 (caméra). Il faut accepter le certificat sur les deux, sinon la vidéo reste noire.
5	Stations sans coordonnées	pepper et imp3d valent None : MARC refuse d'y aller. Il y a actuellement pas assez d'Aruco pour une navigation fiable. A ajouter
6	Zone morte moteur	Toute consigne d'avance inférieure à 50 ne produit aucun mouvement. Peut etre modifier dans le fichier .ino

3. Architecture

Trois couches : l'Arduino Mega gère le temps réel (moteurs, IMU, capteurs), le Raspberry Pi 4 fait tourner ROS 2 et l'interface web, et le LLM est hébergé dans le cloud via Ollama.

Les 8 nœuds ROS 2

Nœud	Rôle
firmware_node	Pont série avec l'Arduino, dans les deux sens
camera_node	Caméra IMX219, détection ArUco, flux vidéo (port 5001)
localization_node	Fusion vision + odométrie → pose (x, y, cap)
navigation_node	Go-to-goal : rejoindre un point
mission_node	Résout la destination et planifie le chemin (Dijkstra)
voice_node	Micro physique → STT → LLM → TTS (désactivé, voir §7)
webbridge_node	Interface web, LLM texte, SSE, relais ROS (port 5000)
led_node	Seul propriétaire de la matrice LED

Règle à respecter

Pour déclencher un déplacement, on publie sur `/mission_station` ou `/mission_point` — jamais directement sur `/nav_goal`. `mission_node` est le seul autorisé à piloter `navigation_node`.

Diagnostic

```
ros2 topic echo /pose          # position temps réel
ros2 topic echo /imu_yaw       # cap (après tare)
ros2 topic echo /nav_status    # statut de mission
ros2 node list
```

4. Communication Raspberry Pi ↔ Arduino

Liaison USB, port /dev/ttyACM0, 115 200 bauds, lignes terminées par un saut de ligne. firmware_node fait tourner un thread dédié à environ 200 Hz : il envoie la consigne moteur, puis lit tout ce qui arrive.

Pi vers Arduino

Trame	Signification
C:<move>:<turn>	Consigne moteur. move = avance (négatif = recul), turn = rotation. Envoyée en continu, même à l'arrêt (C:0:0).
M:0 / M:1	Bascule mode manuel / suivi de ligne (hérité).

La consigne est réémise en boucle, pas envoyée une fois. Il n'y a aucun watchdog côté Arduino : si le Pi cesse d'émettre, le robot garde sa dernière consigne et continue d'avancer.

Arduino vers Pi

Trame	Contenu	Fréquence	Topic
Y:<yaw>	Cap brut IMU (degrés)	10 Hz	/imu_yaw
D:<dist>	Distance cumulée (cm)	10 Hz	/distance
U:<c>:<g>:<d>	Ultrasons (cm), -1 = hors portée	10 Hz	/obstacles
S:<n>	Numéro de station (hérité)	au changement	/station

Les autres lignes émises par l'Arduino (messages de debug) sont ignorées. Les ultrasons latéraux sont filtrés par médiane glissante sur 10 valeurs.

Tester le lien sans ROS

```
screen /dev/ttyACM0 115200
```

Vous devez voir défiler des Y:, D: et U:. Si rien ne sort, le problème est matériel ou de port. Fermez le moniteur série avant de lancer ROS : le port ne s'ouvre qu'une fois.

5. Orientation au démarrage

Section la plus importante du document. Une erreur ici et MARC part dans la mauvaise direction, sans aucun message d'erreur.

Le repère de la salle

- Origine (0, 0) : une intersection de carreaux du sol, matérialisée physiquement.
- Unités : mètres.
- Angles : 0° = direction -Y, +90° = direction +X, sens trigonométrique.

Le cap 0° n'est donc pas « vers le haut du plan » mais vers -Y. Cette convention est reprise partout dans le code.

Pourquoi l'orientation compte

L'IMU BNO085 donne un cap relatif à son propre démarrage, sans référence absolue. firmware_node fait une tare automatique 5 secondes après la première trame IMU :

```
self.IMU_SETTLE_S = 5.0
if self._imu_offset is None:
    if time.time() - self._imu_start >= self.IMU_SETTLE_S:
        self._imu_offset = self._imu_yaw_brut # tare automatique
```

Et localization_node utilise YAW_OFFSET = 0, c'est-à-dire qu'il considère le cap publié comme étant déjà le cap salle.

Conséquence : l'orientation dans laquelle se trouve MARC 5 secondes après le démarrage devient, par définition, le cap 0° de la salle.

Procédure correcte

1. Poser MARC sur le repère de départ.
2. L'orienter face à la direction -Y de la salle.
3. Le laisser parfaitement immobile.
4. Lancer le système.
5. Attendre au moins 5 secondes avant de le toucher.
6. Vérifier : ros2 topic echo /imu_yaw doit afficher une valeur proche de 0.

Re-tarer le cap

Si l'orientation est fausse, placez MARC face au cap 0° puis re-tarez. Trois moyens, au choix :

Moyen	Comment
Interface web	Bouton de re-tare du cap dans l'interface (route /imu_tare). C'est le plus simple sur le terrain.
Voix	« Réinitialise ton cap » (action resetYaw).
Terminal	ros2 topic pub --once /imu_tare std_msgs/Empty "{}"

L'orientation courante devient le nouveau 0°.

Recalage par vision

L'IMU seul dérive. La position absolue vient des marqueurs ArUco fixés aux murs et au plafond. Quand un marqueur connu est visible, la position est recalée (source : vision) ; sinon elle est intégrée depuis la distance parcourue (source : odometrie). Le champ source de /pose indique lequel est actif — si vous restez longtemps en odometrie, la position dérive.

Au démarrage à froid, aucune position n'est connue tant qu'aucun marqueur n'a été vu. Un scan de localisation (rotation sur place par à-coups, abandon après 40 s) permet d'en acquérir une : bouton dans l'interface web, ou « localise-toi » à la voix.

6. Installation

Prérequis : Raspberry Pi 4 avec ROS 2 Jazzy, Arduino Mega 2560 + IDE Arduino, Python 3.9+, daemon Ollama.

Raspberry Pi

```
git clone https://github.com/SALLAH-JP/MARC.git && cd MARC

pip install -r assistantVocale/requirements.txt
pip install opencv-contrib-python pyserial speechrecognition \
    edge-tts pydub pillow flask
sudo apt install ffmpeg mpg123 sox alsa-utils portaudio19-dev

# Matrice LED
git clone https://github.com/hzeller/rpi-rgb-led-matrix
cd rpi-rgb-led-matrix && make build-python PYTHON=$(which python3) && cd ..
sudo setcap 'cap_sys_rawio+ep' $(readlink -f $(which python3))

# Certificat auto-signé (HTTPS obligatoire pour le micro navigateur)
cd web && openssl req -x509 -newkey rsa:4096 -keyout key.pem \
    -out cert.pem -days 365 -nodes && cd ..

ollama signin

source /opt/ros/jazzy/setup.bash
colcon build --packages-select marc_nodes
source install/setup.bash
```

Sans le setcap, led_node se lance mais n'affiche rien.

Arduino

Ouvrir controlMoteur/controlMoteur.ino, installer les bibliothèques FastAccelStepper, IRremote, NewPing, SparkFun BNO08x et PID_v1, vérifier le brochage dans header.h, puis téléverser.

Élément	Pins
Moteur gauche (STEP/DIR/ENA)	7 / 5 / 3
Moteur droit (STEP/DIR/ENA)	8 / 6 / 4
BNO085 (SPI)	CS 47, INT A4, RST A5
Ultrasons centre / gauche / droite	22-23 / 24-25 / 26-27
Capteurs ligne G / C / D	49 / 40 / 48
Récepteur IR	46

Chemins codés en dur

webbridge_node.py, voice_node.py, led_node.py et camera_node.py contiennent ROOT = "/home/pi/MARC". Si le dépôt est ailleurs, créez un lien symbolique plutôt que de modifier les quatre fichiers :

```
sudo ln -s /chemin/reel/vers/MARC /home/pi/MARC
```

Calibration caméra

Sans calibration, les marqueurs sont détectés mais sans distance ni angle, donc pas de localisation. Lancer `vision/calibrate_camera.py` (damier) pour produire `vision/camera_calibration.npz`. Deux constantes l'accompagnent dans `camera_node.py` : `MARKER_SIZE_M` = 0.077 (taille imprimée du marqueur) et `DISTANCE_CORRECTION` = 0.738 (correction du biais mesuré). Si vous réimprimez des marqueurs d'une autre taille, changez la première et remesurez la seconde.

7. Démarrage et utilisation

Checklist

7. Batteries chargées (deux circuits : moteurs et Pi).
8. Arduino relié en USB, aucun moniteur série ouvert.
9. MARC posé sur l'origine, orienté cap 0°, immobile.
10. Réseau Wi-Fi disponible (LLM cloud et STT Google).
11. Lancer, puis ne pas toucher pendant 5 secondes.

Lancement

```
source /opt/ros/jazzy/setup.bash && source install/setup.bash  
ros2 launch marc_nodes marc.launch.py use_voice:=false
```

Argument	Défaut	Effet
port:=/dev/ttyACM1	/dev/ttyACM0	Port série Arduino
style:=1	2	Style d'yeux au démarrage
use_voice:=false	true	Coupe le micro physique
use_led:=false	true	Coupe la matrice LED
use_vision:=false	true	Coupe caméra, localisation, navigation, mission

Arrêt : Ctrl+C. Les nœuds coupent les moteurs et éteignent la matrice en sortant.

Interface web

Adresse : <https://<IP-du-Pi>:5000>

Il y a deux serveurs HTTPS : le port 5000 (interface) et le port 5001 (caméra). Visitez <https://<IP>:5001/health> une fois et acceptez le certificat, sinon le flux vidéo reste noir alors que le reste fonctionne. C'est le symptôme le plus souvent mal diagnostiqué.

L'interface propose : la carte avec la position live (un clic sur une station lance un déplacement), la pose temps réel avec sa source, le flux caméra et les marqueurs détectés, le bouton Push-To-Talk, les boutons de re-tare et de localisation, et le journal. Les mises à jour arrivent par SSE. Les réponses de MARC sont diffusées phrase par phrase : il commence à parler avant que le LLM ait fini.

Le micro physique

Le micro embarqué est de qualité insuffisante : la détection du mot de réveil est peu fiable et les transcriptions sont mauvaises. En pratique voice_node n'est pas lancé et on utilise le Push-To-Talk de l'interface web, qui capte l'audio avec le micro du téléphone ou du PC.

Dans marc.launch.py, use_voice est encore déclaré avec default_value="true". Passez use_voice:=false à chaque lancement, ou changez cette valeur par défaut.

Si vous remplacez le micro : assistantVocale/voiceAssistant.py utilise sr.Microphone(device_index=1) codé en dur, à deux endroits (calibrate_mic et listen_once). Listez vos périphériques avec arecord -l et ajustez l'index aux deux endroits. Mot de réveil : « salut marc ». Mot d'arrêt : « merci ».

Actions disponibles

Action	Paramètres	Effet
moveTo	destination	Navigation vers une station
goTo	x, y (mètres)	Navigation vers un point libre
stopNav	—	Arrêt de la mission
resetYaw	—	Re-tare du cap
localize	—	Scan de localisation
moveForward / moveBackward	temps (s)	Avance / recul temporisé
turnLeft / turnRight	temps (s)	Rotation temporisée
changeEyes	style (1-3)	Change le style d'yeux
shutdown	—	Arrêt et extinction de la matrice

8. Modifier la carte

Tout est dans `src/marc_nodes/marc_nodes/maps.py`. C'est le seul fichier à toucher pour la géographie de la salle. Recompilez après chaque modification.

Ajouter ou déplacer une station

Mesurez au mètre ruban depuis l'origine, en mètres. Mesurez le point où MARC doit s'arrêter, pas le robot exposé (comptez environ 50 cm de dégagement).

```
STATION_POS = {
    "nao":      (-1.6, 0),
    "pepper":   (2.4, 3.1),    # renseigné
    "imp3d":    None,          # toujours à mesurer
}

STATION_HEADING = {          # cap à adopter en arrivant
    "nao":      90,
    "pepper":   180,
    "imp3d":    None,          # None = pas de contrainte
}
```

Une station à None fait que `mission_node` refuse le trajet plutôt que d'inventer une trajectoire.

Pour une station entièrement nouvelle, trois ajouts supplémentaires : `DESTINATION_MAP` dans `webbridge_node.py` (traduction du nom sorti par le LLM), la liste des destinations dans `assistantVocale/modelfile.txt`, et l'interface web si vous voulez qu'elle apparaisse sur la carte.

Ajouter un point de passage

Les waypoints forment le graphe de chemins sûrs sur lequel `mission_node` calcule le plus court chemin.

```
WAYPOINTS = { "origin": (0.0, 0.0), "B": (-1.2, 0), "J": (1.5, 3.0) }
EDGES      = [ ("origin", "B"), ("H", "J") ]
```

- Ne déclarez une arête que si le segment en ligne droite est physiquement libre. Le système ne le vérifie pas.
- Les arêtes sont bidirectionnelles, le coût est la distance euclidienne.
- Le graphe doit être assez dense : les petits segments d'entrée et de sortie (de MARC vers le premier nœud, du dernier nœud vers la station) ne sont pas planifiés.
- Graphe vide ou pose inconnue : repli sur un trajet direct, sans garantie d'évitement.

Ajouter un marqueur ArUco

```
LANDMARKS_MAP = { 0: (-2.07, 0.00), 6: (3.10, 2.20) }
```

- Dictionnaire `DICT_4X4_50` : les IDs vont de 0 à 49.
- Taille imprimée : 77 mm de côté. Tous les marqueurs doivent faire la même taille.
- Position à mesurer au centre du marqueur.
- Un marqueur absent de `LANDMARKS_MAP` est ignoré pour la localisation.
- Quand plusieurs marqueurs sont visibles, les positions sont moyennées.

Posez-les aux murs et au plafond, répartis pour qu'il y en ait toujours un dans le champ. C'est ce qui évite les longues phases d'odométrie pure, où la position dérive.

9. Modifier les yeux

led_node est le seul propriétaire de la matrice, ce qui évite les conflits d'accès GPIO. Tout nœud qui veut afficher une expression publiée sur /led_expression :

```
ros2 topic pub --once /led_expression std_msgs/String "{data: 'love'}"
ros2 topic pub --once /led_expression std_msgs/String "{data: 'idle:neutral'}"
ros2 topic pub --once /led_expression std_msgs/String "{data: 'style:3'}"
```

Expressions existantes : neutral, blink, suspicious, sad, love, disappear. EyeManager joue l'animation de repos en boucle et cligne spontanément toutes les 3 à 7 secondes.

Ajouter une expression

Trois fichiers, dans cet ordre.

12. Créer le GIF en 64×32. Le script matrixLed/SplitEyes.py extrait des GIF depuis une vidéo source (liste de tuples nom, début, fin). Placez le résultat dans TOUS les dossiers style1/, style2/, style3/, sinon l'expression disparaît au changement de style.
13. Déclarer l'animation dans le dictionnaire ANIMATIONS de matrixLed/eye_manager.py.
14. Autoriser le nom dans KNOWN_ANIMATIONS de led_node.py. Sans cette étape, le message est rejeté avec « Animation inconnue » — c'est l'oubli classique.

```
# eye_manager.py
ANIMATIONS = { ..., "angry": "angry.gif" }

# led_node.py
KNOWN_ANIMATIONS = { ..., "angry" }
```

Test rapide hors ROS, pendant la mise au point : `cd matrixLed && python3 eye_manager.py`

Ajouter un style complet

Créez matrixLed/style4/ contenant tous les GIF listés dans ANIMATIONS. Il devient accessible par style:4, ou par l'argument de lancement style:=4. Un GIF manquant génère un avertissement au chargement et l'animation ne joue pas.

Réglages du panneau

Dans led_node.py : led_rgb_sequence = "RBG" n'est pas une faute de frappe, ce panneau a le vert et le bleu inversés. Si vous changez de panneau et que les couleurs sont fausses, c'est ici. Luminosité : brightness = 75.

10. Réglages de navigation

Dans navigation_node.py :

Constante	Valeur	Rôle
GOAL_TOLERANCE_M	0.25	Rayon d'arrivée (m)
ANGLE_TOLERANCE	15	Au-delà : rotation sur place avant d'avancer (°)
HEADING_TOLERANCE	8	Tolérance sur le cap final (°)
KP_TURN	3.0	Gain de correction de cap en roulant
TURN_SPEED	150	Consigne de rotation sur place
FORWARD_SPEED	300	Consigne d'avance
MAX_LOST	20	Cycles sans pose avant abandon
SCAN_TIMEOUT_S	40	Abandon du scan de localisation (s)

Symptôme	Piste
MARC oscille autour de sa trajectoire	Baisser KP_TURN
Corrige trop mollement, part en biais	Monter KP_TURN
S'arrête trop loin de la station	Baisser GOAL_TOLERANCE_M
Abandonne trop vite (« Localisation perdue »)	Monter MAX_LOST ou ajouter des marqueurs
Le scan tourne trop vite pour détecter	Baisser SCAN_TURN_SPEED

Ne descendez pas FORWARD_SPEED sous 50 : le firmware Arduino coupe les moteurs sous ce seuil.

À vérifier sur le matériel : les actions manuelles turnLeft et turnRight de webbridge_node utilisent une convention de signe opposée à celle de navigation_node. Confirmez le sens réel avant de vous y fier.

11. Dépannage

Symptôme	Cause probable	Solution
[Arduino] Non connecté	Mauvais port ou moniteur série ouvert	ls /dev/ttyACM* puis port:=/dev/ttyACM1
MARC part dans la mauvaise direction	Tare faite dans la mauvaise orientation	Re-tarer depuis l'interface web (§5)
/pose ne publie rien	Aucun marqueur jamais vu	Lancer un scan de localisation
source: odometrie en permanence	Marqueurs hors champ ou non calibrés	Vérifier la calibration, ajouter des marqueurs
Position juste mais dérive vite	Roues qui patinent, recalage rare	Densifier les marqueurs
Vidéo noire, reste de l'interface OK	Certificat du port 5001 non accepté	Ouvrir https://<IP>:5001/health
Micro refusé par le navigateur	Page en HTTP ou certificat refusé	Passer par https://
rgbmatrix non disponible	Bibliothèque ou setcap manquants	Recompiler, refaire le setcap
Animation inconnue : X	Nom absent de KNOWN_ANIMATIONS	Voir §9
MARC ne comprend rien	Ollama arrêté ou pas d'internet	ollama serve ; curl localhost:11434/api/tags
Le robot ne bouge pas	Consigne sous la zone morte (< 50)	Monter la consigne
Import échoue au lancement	ROOT ne pointe pas sur le dépôt	Lien symbolique vers /home/pi/MARC
Modification sans effet	Pas recompilé	colcon build --packages-select marc_nodes

12. Aide-mémoire

```
# Lancement normal
ros2 launch marc_nodes marc.launch.py use_voice:=false

# Recompile après toute modification Python
colcon build --packages-select marc_nodes && source install/setup.bash

# Diagnostic
ros2 topic echo /pose
ros2 topic echo /imu_yaw
ros2 topic echo /nav_status

# Actions manuelles
ros2 topic pub --once /imu_tare std_msgs/Empty "{}"
ros2 topic pub --once /nav_localize std_msgs/Empty "{}"
ros2 topic pub --once /mission_station std_msgs/String "{data: 'nao'}"
ros2 topic pub --once /cmd_motor std_msgs/Int32MultiArray "{data: [0, 0]}" # STOP

# Interface : https://<IP>:5000      Caméra : https://<IP>:5001/health
```

Fichiers les plus souvent modifiés

Fichier	Pour quoi
marc_nodes/maps.py	Stations, waypoints, marqueurs
marc_nodes/navigation_node.py	Constantes de réglage
marc_nodes/localization_node.py	YAW_OFFSET
marc_nodes/webbridge_node.py	Actions, interface, LLM
marc_nodes/led_node.py	KNOWN_ANIMATIONS
matrixLed/eye_manager.py	Dictionnaire ANIMATIONS
assistantVocale/modelfile.txt	Prompt système, actions, connaissances
controlMoteur/header.h	Brochage, PID
launch/marc.launch.py	Arguments de lancement